Distillery: Transparent Method Selection for Smaller Models

Anthropic 2

July 2026

Abstract

Distillery is a high-level system for deciding whether existing traces and synthetic examples should become a smaller portable model. The workflow is Curate, Synthesize, Train, and Prove. A pure planner checks model revisions, tokenizer compatibility, license and output-use constraints, leakage, memory, runtime, and cost before any billable training begins. The first release implements sequence distillation and exact same-tokenizer logit distillation, uses one sealed manifest per finite local or SageMaker job, and produces immutable model and proof artifacts. The system can recommend no training and does not treat job completion as success.

1 Product contract

The public resources are `Dataset`, asynchronous `DistillationRun`, immutable `ModelArtifact`, and immutable `ProofReport`. Each resource is versioned and hash-addressed. Run states are monotonic, duplicate creation uses an idempotency key, and cancellation races are resolved without inventing partial success.

The high-level Python path is:

```
distillery = Distillery(api_key=os.environ["DISTILLERY_API_KEY"])
dataset = distillery.datasets.create("./finance_world.jsonl")
run = distillery.distill(dataset, recipe="auto").wait()
```

This compact surface does not erase the underlying decision. The resolved recipe, model pair, fingerprints, expected cost and memory range, capability checks, and errors remain visible.

2 Planning before execution

`plan_distillation()` is pure and non-billable. It returns pinned model revisions and fingerprints, dataset and split identity, recipe capability, tokenizer and chat-template compatibility, license and output-use gates, leakage checks, a memory probe, wall-time and cost ranges, and typed errors.

The planner refuses unpinned revisions, incompatible tokenizers, unresolved license gates, contaminated evaluation data, or cost estimates above the run ceiling. A requested recipe is never silently downgraded. Catalog methods that are not implemented return `RECIPE_NOT_IMPLEMENTED`.

3 Curate and Synthesize

Curate validates imported traces, records provenance, rejects malformed task outputs, isolates groups and OOD regimes, and freezes hashes. Synthesize preserves valid imported responses and

requests new labels only for missing, rejected, or deliberately augmented examples. Teacher calls can therefore be zero when the existing response set is complete.

This teacher-sparing behavior is part of the method resolver rather than a marketing promise. Counts of imported, oracle, teacher-generated, and rejected labels are carried into the run manifest and proof package.

4 Train

The release supports `sequence.v1`, `logit.v1`, and transparent `auto`. Sequence distillation performs completion-only training over validated text responses. Logit distillation performs chunked exact full-vocabulary forward KL at output positions, with a frozen teacher, temperature scaling, and optional hard-target mixing. A matched CE-only mode differs from logit KD only in objective fields.

One sealed manifest maps to one finite training job. The local backend and SageMaker backend execute the same entrypoint. The AWS path uses a single GPU instance, a bounded maximum runtime, explicit cost ceilings, project and run tags, unique artifact prefixes, and terminal-state verification. It creates no endpoint, notebook, or persistent GPU resource.

5 Prove

Proof runs against immutable predictions, systems profiles, manifests, and cost records. It compares rules, teacher, frozen base student, a cheap off-the-shelf model, oracle SFT, sequence KD, logit KD, and matched CE. The report contains deterministic task metrics, paired clustered confidence intervals, slices, quality retention, latency, throughput, VRAM, GPU use, gross cost, and utilization-sensitive break-even.

Allowed final statuses are `proved`, `do_not_distill`, `failed_quality`, `failed_economics`, and `insufficient_evidence`. The first failed gate is recorded. A missing baseline, seed, cost record, or required arm prevents a proved status.

6 Implementation result

The released system completed the four-stage workflow for TinyFable. It created a frozen synthetic finance dataset, disclosed its recipe resolution, executed the sealed training manifest through the finite job backend, reloaded the resulting portable artifact, and produced an immutable proof report. TinyFable received `proved`; the same interfaces also exercised non-success statuses through frozen proof fixtures.

Stage	Durable output	Release result
Curate	Dataset and split hashes	Complete
Synthesize	Label provenance ledger	Complete
Train	Model artifact and checksums	Complete
Prove	Immutable proof report	`proved`

Table 1: TinyFable exercised the complete Distillery path.

7 Scope

Distillery is a scientific experiment and reusable platform, not a production traffic system. The release excludes serving, endpoint deployment, traffic routing, live trace capture, continuous training, customer data, privacy and redaction controls, multitenancy, RBAC, billing, and arbitrary provider adapters.